TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI

BÁO CÁO THỰC HÀNH

MÔN: TƯƠNG TÁC NGƯỜI - ROBOT RBE3053

BÀI THỰC HÀNH SỐ 6

Giao tiếp phi ngôn ngữ: điều khiển TurtleBot 4 bằng cử chỉ tay

Sinh viên thực hiện: **Lê Trọng Nghĩa – 23020754**

Giảng viên phụ trách: **PGS.TS. Hoàng Văn Xiêm**
CN. Nguyễn Quốc Bảo

MỤC LỤC

1. Mục tiêu và môi trường 2

2. Sơ đồ kiến trúc hệ thống 2

3. Các node, topic và luồng xử lý 3

3.1. Node và topic 4

3.2. Luồng xử lý trong node nhận diện 4

3.3. Máy trạng thái và nguyên lý điều khiển robot 5

4. Bảng ánh xạ cử chỉ tay và hành vi robot 6

4.1. Đáp ứng các yêu cầu an toàn và chống nhiễu 6

5. Kết quả thực nghiệm 7

6. Nhận xét và giới hạn 9

Nội dung nộp	Liên kết
Video demo (Google Drive, công khai)	(chưa điền – docs/report_info.yaml)
GitHub repository (Public)	(chưa điền – docs/report_info.yaml)

Bảng 0 – Liên kết video demo và mã nguồn.

1. Mục tiêu và môi trường

Bài thực hành xây dựng kênh giao tiếp phi ngôn ngữ giữa người và robot: người dùng ra cử chỉ tay trước webcam laptop, hệ thống nhận diện cử chỉ bằng mô hình học máy MediaPipe Gesture Recognizer và điều khiển TurtleBot 4 trong Gazebo thực hiện hành vi tương ứng qua ROS 2. Robot chỉ thực hiện hành vi sau cử chỉ xác nhận, không dùng bàn phím trong lúc demo, có cơ chế chống nhận diện nhiễu và luôn dừng khi có tín hiệu dừng hoặc mất tín hiệu điều khiển.

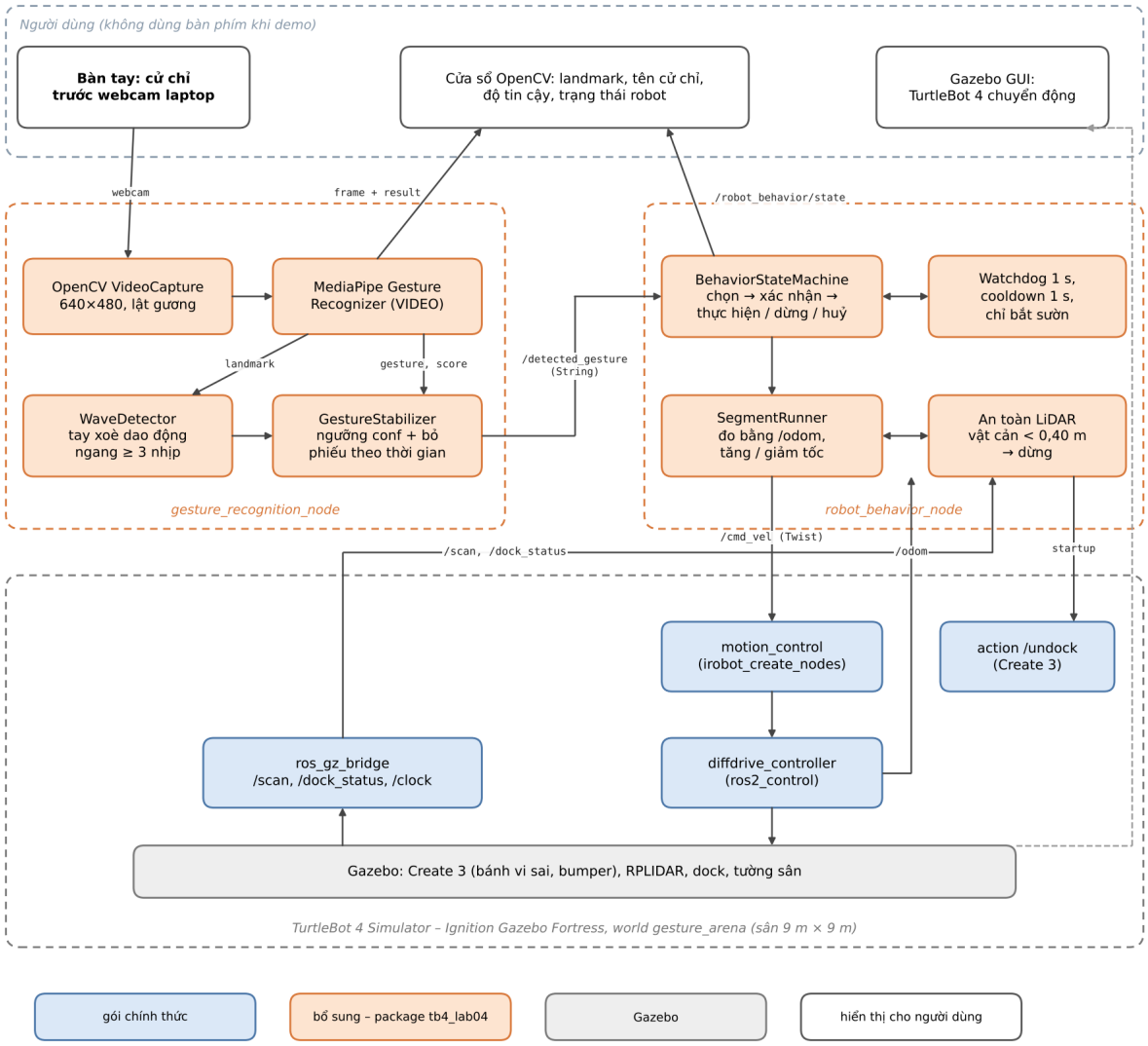
Môi trường: Ubuntu 22.04, ROS 2 Humble, Ignition Gazebo Fortress, TurtleBot 4 Simulator (Create 3 + RPLIDAR), Python 3.10, OpenCV 4.10, MediaPipe 0.10.21 (NumPy 1.26 để không xung đột với `cv_bridge` của Humble). Mã nguồn nằm trong package `tb4_lab04` của các bài trước, bổ sung hai node `gesture_recognition_node`, `robot_behavior_node`, module logic `gesture_logic.py` (có kiểm thử đơn vị), ba launch file và world `gesture_arena` – sân phẳng 9 m × 9 m không vật cản để các hành vi không va chạm.

File	Vai trò
`launch/gesture_sim.launch.py`	Phần 1: Gazebo + TurtleBot 4 trong world gesture_arena
`launch/gesture.launch.py`	Phần 2 + 3: hai node cử chỉ, tham số config/gesture.yaml
`launch/gesture_demo.launch.py`	tất cả trong một lệnh (node cử chỉ bật khi mô phỏng sẵn sàng)
`scripts/gesture_recognition_node.py`	webcam → MediaPipe → lọc nhiễu → /detected_gesture
`scripts/robot_behavior_node.py`	/detected_gesture → máy trạng thái → /cmd_vel
`scripts/gesture_logic.py`	WaveDetector, GestureStabilizer, BehaviorStateMachine, SegmentRunner
`scripts/webcam_test.py, check_cmd_vel.py`	Phần 1: hiển thị webcam (OpenCV); kiểm tra robot nhận /cmd_vel
`worlds/gesture_arena.sdf, models/`	sân mô phỏng; model gesture_recognizer.task (float16, 8,4 MB)

Bảng 1 – Các file chính của Bài 6.

2. Sơ đồ kiến trúc hệ thống

Hệ thống gồm ba tầng (Hình 1). Tầng người dùng: bàn tay trước webcam; người dùng theo dõi kết quả trên cửa sổ OpenCV (tên cử chỉ, độ tin cậy, trạng thái robot) và trên Gazebo. Tầng ứng dụng gồm hai node ROS 2 tự viết, nối với nhau bằng topic `/detected_gesture` (`std_msgs/String`). Tầng mô phỏng là TurtleBot 4 chính thức: `motion_control` của Create 3 nhận `/cmd_vel`, `diffdrive_controller` phát `/odom`, `ros_gz_bridge` đưa `/scan` và `/dock_status` từ Gazebo sang ROS 2.



Hình 1 – Kiến trúc hệ thống (cam: phần tự viết trong package tb4_lab04; xanh: gói chính thức).

3. Các node, topic và luồng xử lý

3.1. Node và topic

Node	Subscribe	Publish	Vai trò
`gesture_recognition_node`	`/robot_behavior/state`	`/detected_gesture` `/detected_gesture/info`	đọc webcam bằng OpenCV, nhận diện bằng MediaPipe, suy ra vẫy tay, lọc nhiễu, hiển thị
`robot_behavior_node`	`/detected_gesture` `/odom, /scan` `/dock_status`	`/cmd_vel` `/robot_behavior/state`	máy trạng thái, thực hiện hành vi, watchdog, an toàn, undock (action client /undock)
`motion_control (Create 3)`	`/cmd_vel`	–	áp giới hạn an toàn, chuyển lệnh tới bộ điều khiển bánh
`diffdrive_controller`	–	`/odom, TF`	odometry bánh xe (dùng để đo quãng đường / góc quay)

Bảng 2 – Các node chính và topic vào/ra.

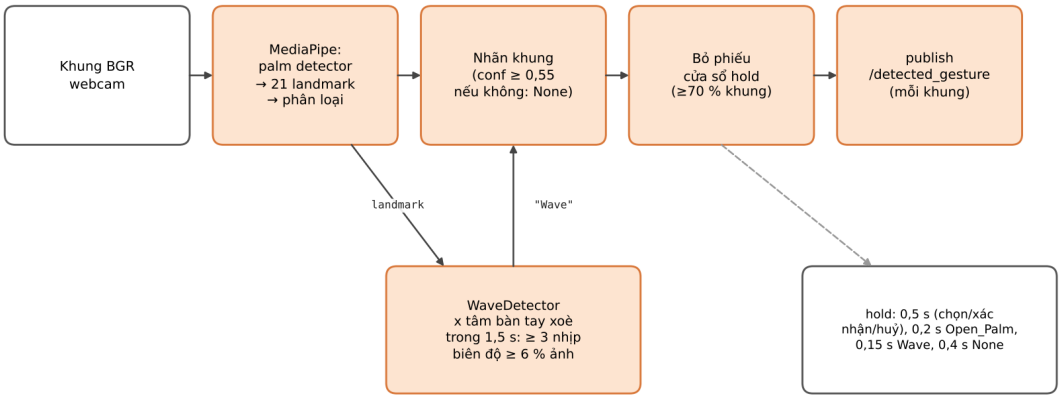
Topic / action	Kiểu	Ý nghĩa
`/detected_gesture`	`std_msgs/String`	tên cử chỉ ĐÃ LỌC (Wave, Pointing_Up, Victory, Thumb_Up, Open_Palm, Closed_Fist, Thumb_Down, ILoveYou, None), gửi mỗi khung ~30 Hz
`/detected_gesture/info`	`std_msgs/String (JSON)`	nhãn thô của khung, độ tin cậy, tỉ lệ phiếu, FPS
`/robot_behavior/state`	`std_msgs/String (JSON)`	trạng thái, hành vi đã chọn / đang chạy, tiến độ
`/cmd_vel`	`geometry_msgs/Twist`	lệnh vận tốc 20 Hz khi đang thực hiện; 5 lệnh 0 khi dừng
`/odom`	`nav_msgs/Odometry`	đo tiến độ từng đoạn chuyển động, phát hiện robot bị kẹt
`/scan`	`sensor_msgs/LaserScan`	dừng khi có vật cản < 0,40 m theo hướng đang chạy
`/dock_status, /undock`	`irobot_create_msgs`	robot ở dock lúc khởi động → tự undock + ra giữa sân 1 m

Bảng 3 – Các topic và action.

3.2. Luồng xử lý trong node nhận diện

Mỗi khung hình đi qua chuỗi xử lý trên Hình 2. OpenCV đọc webcam ở luồng riêng và luôn lấy khung mới nhất (bộ đệm 1 khung) nên không bị trễ dồn khi xử lý chậm hơn tốc độ camera. Ảnh được lật gương rồi đưa vào MediaPipe Gesture Recognizer chạy ở chế độ `VIDEO`: mô hình phát hiện lòng bàn tay định vị bàn tay, mô hình landmark trả 21 điểm khớp 3D, bộ phân loại cử chỉ (mạng nơ-ron nhỏ trên vector landmark) trả một trong 7 lớp kèm điểm tin cậy. Chế độ VIDEO dùng lại vị trí tay của khung trước nên không phải chạy bộ phát hiện ở mọi khung.

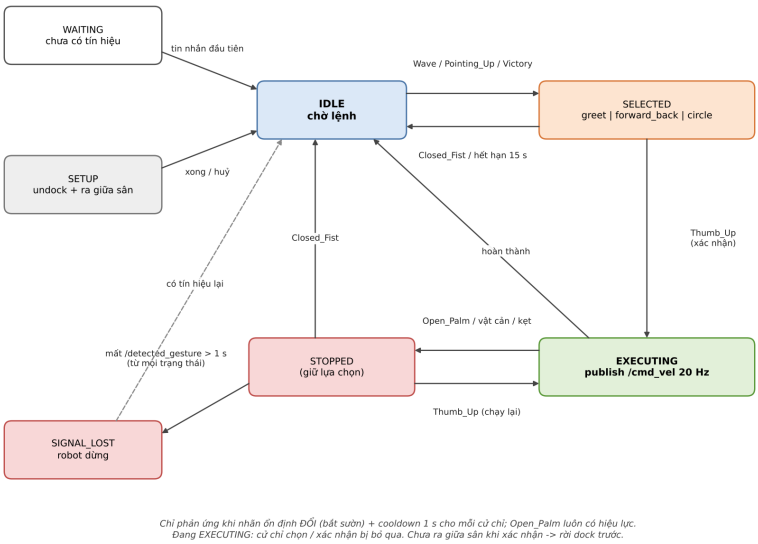
Vẫy tay không có trong 7 lớp của mô hình nên được suy ra bằng luật trên landmark (`WaveDetector`): theo dõi hoành độ tâm lòng bàn tay ở các khung có bàn tay xoè; trong 1,5 s gần nhất có ít nhất 3 nhịp qua lại, mỗi nhịp dài ít nhất 6 % bề ngang ảnh thì coi là vẫy. Khung có điểm tin cậy dưới 0,55 được coi là "None". Cuối cùng `GestureStabilizer` bỏ phiếu trong cửa sổ thời gian: một nhãn chỉ được công bố khi chiếm ít nhất 70 % số khung trong 0,5 s gần nhất (0,2 s với Open_Palm để dừng nhanh). Nhãn ổn định được publish ở mọi khung; nếu camera lỗi thì node ngừng publish.



Hình 2 – Xử lý một khung hình trong gesture_recognition_node.

3.3. Máy trạng thái và nguyên lý điều khiển robot

Node hành vi là một máy trạng thái (Hình 3). Cử chỉ chọn (vẫy tay, một ngón, hai ngón) chỉ ghi nhớ hành vi; ngón cái mới chuyển sang `EXECUTING`. Khi thực hiện, mỗi hành vi là một chuỗi đoạn chuyển động (vận tốc thẳng v, vận tốc góc ω, đích là quãng đường hoặc góc). `SegmentRunner` publish `/cmd_vel` 20 Hz và đo tiến độ bằng `/odom` (không hẹn giờ): đoạn tiến/lùi kết thúc khi robot đi đủ 0,35 m, đoạn quay khi yaw tích luỹ đủ góc, quỹ đạo tròn khi quay đủ 360° với v = 0,15 m/s, ω = v/r = 0,3 rad/s. Lệnh được tăng tốc dần và giảm tốc gần đích theo $v \le \sqrt{(2 \cdot a \cdot s)}$ (có bù trễ đáp ứng 0,15 s) nên robot không vượt đích; các hành vi đối xứng nên robot kết thúc gần đúng chỗ và hướng ban đầu.



Hình 3 – Máy trạng thái của robot_behavior_node.

4. Bảng ánh xạ cử chỉ tay và hành vi robot

Cử chỉ	Nhãn (MediaPipe)	Hành vi robot	Thông số
Vẫy tay	`Wave (suy ra từ Open_Palm + chuyển động)`	chọn "chào lại": xoay trái–phải	±0,6 rad, 1 rad/s, 2 lần; ~5 s
Giơ 1 ngón tay	`Pointing_Up`	chọn tiến–lùi 4 lần	0,35 m, 0,2 m/s; ~20 s
Giơ 2 ngón tay	`Victory`	chọn quỹ đạo tròn	r = 0,5 m, 0,15 m/s, 1 vòng; ~21 s
Giơ ngón cái	`Thumb_Up`	xác nhận, thực hiện hành vi đã chọn	chỉ khi đã chọn
Xoè bàn tay	`Open_Palm`	dừng ngay (giữ lựa chọn)	giữ 0,2 s; không cooldown
Nắm tay	`Closed_Fist`	huỷ hành vi, xoá lựa chọn, về chờ	–
(thêm)	`Thumb_Down, ILoveYou`	chỉ hiển thị / publish	tổng 8 cử chỉ

Bảng 4 – Ánh xạ cử chỉ → hành vi.

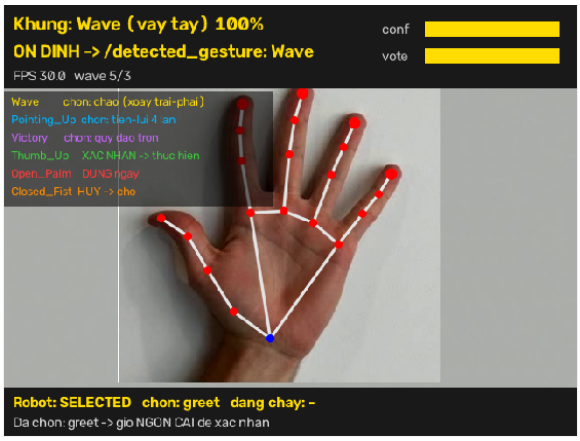
Đề bài vừa ghi "vẫy tay → robot xoay trái–phải để chào lại" vừa yêu cầu "robot chỉ thực hiện hành vi sau khi nhận được cử chỉ xác nhận". Mặc định `wave_requires_confirm: true` thoả cả hai: vẫy tay chọn hành vi chào, ngón cái xác nhận; đặt `false` nếu muốn robot chào lại ngay.

4.1. Đáp ứng các yêu cầu an toàn và chống nhiễu

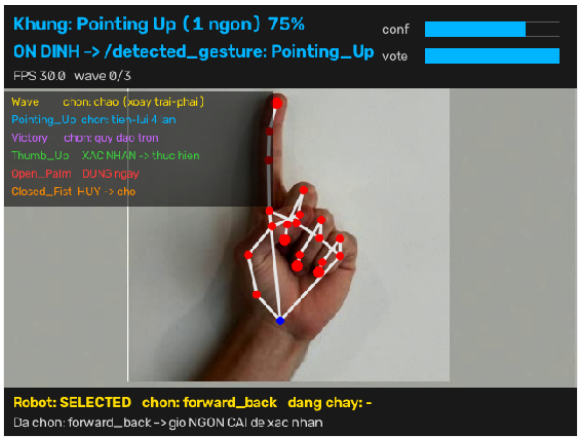
Yêu cầu	Cách thực hiện
Chỉ chạy sau xác nhận	cử chỉ chọn chỉ ghi nhớ; Thumb_Up khi chưa chọn bị bỏ qua; chạy xong xoá lựa chọn; lựa chọn hết hạn sau 15 s
Không dùng bàn phím	mọi lệnh qua /detected_gesture; undock và ra giữa sân tự động
Chống nhiễu kích hoạt liên tục	ngưỡng tin cậy 0,55; bỏ phiếu ≥ 70 % trong 0,5 s; chỉ phản ứng khi nhãn ổn định ĐỔI (giữ ngón cái lâu vẫn chỉ 1 lệnh); cooldown 1 s mỗi cử chỉ; đang chạy thì bỏ qua cử chỉ chọn / xác nhận
Dừng khi có tín hiệu dừng	Open_Palm ưu tiên cao nhất → Twist 0 ngay, gửi lặp 5 chu kỳ, huỷ hành vi
Dừng khi mất tín hiệu	> 1 s không có /detected_gesture (node nhận diện tắt, camera lỗi) → dừng, huỷ (SIGNAL_LOST); Ctrl+C node hành vi → gửi Twist 0 trước khi thoát
Không va chạm	sân 9 m × 9 m trống; hành vi nằm trong bán kính 1,05 m quanh điểm bắt đầu (cách tường ≥ 2,5 m); LiDAR: vật cản < 0,40 m theo hướng chạy → dừng; ra lệnh mà odom không đổi 4 s → dừng (kẹt); bước rời dock bị huỷ (còn trong dock hoặc mới lùi ra nửa chừng) → xác nhận đầu tiên sẽ rời dock / quay ra xa dock và ra giữa sân trước

Bảng 5 – Yêu cầu của đề và cách đáp ứng.

5. Kết quả thực nghiệm



(a) Wave

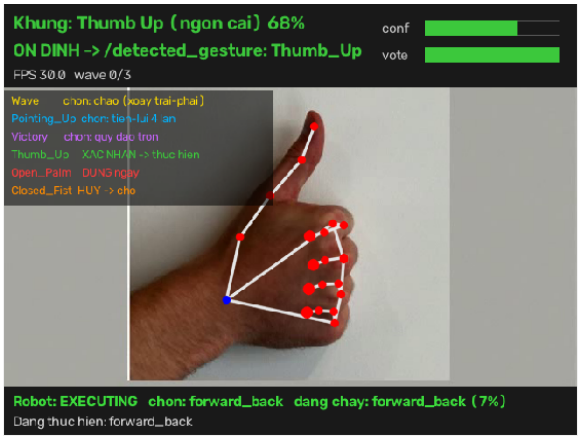


(b) Pointing_Up

Hình 4 – Cửa sổ nhận diện: landmark, tên cử chỉ, độ tin cậy, nhãn ổn định và trạng thái robot (ảnh kiểm thử trong môi trường ảo).

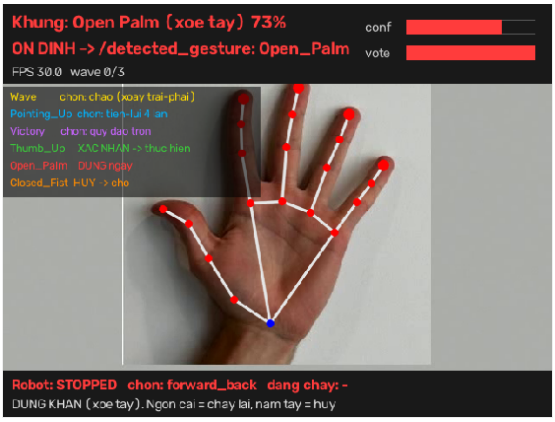


(c) Victory

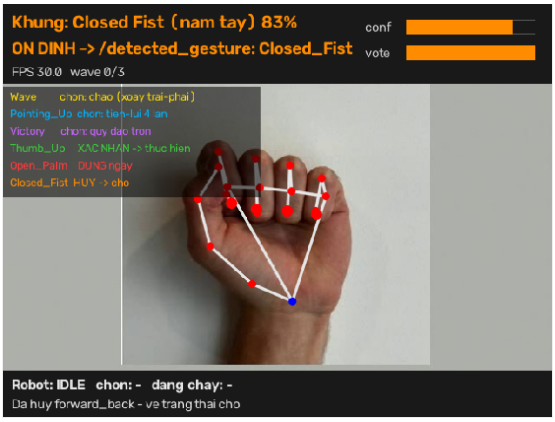


(d) Thumb_Up

Hình 5 – Cửa sổ nhận diện: landmark, tên cử chỉ, độ tin cậy, nhãn ổn định và trạng thái robot (ảnh kiểm thử trong môi trường ảo).



(e) Open_Palm

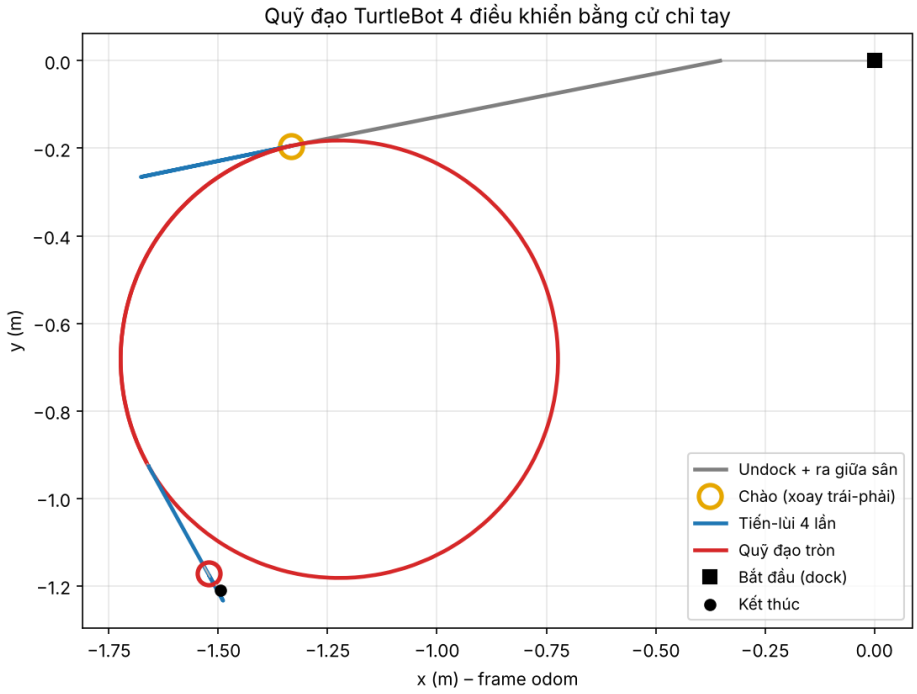


(f) Closed_Fist

Hình 6 – Cửa sổ nhận diện: landmark, tên cử chỉ, độ tin cậy, nhãn ổn định và trạng thái robot (ảnh kiểm thử trong môi trường ảo).

CHƯA CÓ ẢNH
chép ảnh chụp màn hình vào
`docs/report/fig_user/gazebo_gesture.png`

Hình 7 – Gazebo và cửa sổ webcam khi robot đang thực hiện hành vi.



Hình 8 – Quỹ đạo robot (odom) theo từng hành vi, vẽ bằng plot_gesture_run.py (ảnh kiểm thử trong môi trường ảo).

Ngoài thử nghiệm với webcam, hệ thống được kiểm thử tích hợp tự động: TurtleBot 4 Simulator chạy thật trong Gazebo (máy ảo không GPU, real-time factor chỉ 0,06), node hành vi nhận chuỗi cử chỉ giống hệt node nhận diện gửi, script kiểm tra đợi từng chuyển trạng thái và đo vận tốc / vị trí robot từ `/odom`. Tất cả các bước đều đạt (Bảng 6); số đo lấy từ `odom.csv` của các lần chạy đó.

Kịch bản (tự động, Gazebo)	Kết quả đo
Khởi động ở dock → undock + chạy thẳng 1 m	quãng đường 0,999 m, sau đó IDLE
Ngón cái khi chưa chọn	bỏ qua, robot đứng yên
Vẫy tay → ngón cái (chào)	xoay +34,9° / −34,2°, kết thúc lệch hướng 0,0°, không dịch chuyển
1 ngón → ngón cái (giữ ngón cái thêm 3 s)	chỉ 1 lần chạy; 4 lần tiến–lùi 0,34 m, về lệch 1 mm
2 ngón → ngón cái → xoè tay ở 30 %	STOPPED < 0,6 s, vận tốc về 0; ngón cái → chạy lại
Quỹ đạo tròn (sau khi chạy lại)	đường kính 1,00 m (r = 0,5 m), điểm cuối cách điểm đầu 1,1 cm
1 ngón → ngón cái → nắm tay giữa chừng	dừng, IDLE, xoá lựa chọn
Đang chạy thì ngừng gửi /detected_gesture	SIGNAL_LOST sau ~1 s, robot dừng (vận tốc 0)
Ctrl+C node hành vi	gửi Twist 0 rồi thoát sạch
Xoè tay khi robot mới lùi khỏi dock 0,2 m (còn quay mặt vào dock) → 2 ngón → ngón cái	robot quay ra xa dock, chạy ra giữa sân rồi mới vòng tròn; không lúc nào tiến lại gần dock (8/8 bước)

Bảng 6 – Kiểm thử tích hợp tự động trong môi trường ảo.

6. Nhận xét và giới hạn

MediaPipe nhận ổn định các cử chỉ tĩnh rõ ràng (nắm tay, xoè tay, ngón cái, một ngón, hai ngón) khi đủ sáng và bàn tay cách webcam khoảng 0,4–1 m; độ tin cậy giảm khi bàn tay nghiêng, bị che hoặc nền lẫn màu da. Bộ lọc thời gian làm chậm phản hồi khoảng 0,2–0,5 s nhưng loại được các khung nhận nhầm lẻ tẻ. Vẫy tay là luật tự đặt trên landmark nên phụ thuộc biên độ, tốc độ vẫy và FPS (cần khoảng 10 FPS trở lên – khi máy quá tải, chuyển động bị lấy mẫu thiếu và không nhận được); vì vẫy tay bắt đầu bằng bàn tay xoè, vẫy khi robot đang chạy sẽ làm robot dừng – chấp nhận để lệnh dừng luôn được ưu tiên.

Robot chạy vòng hở theo odometry (không dùng bản đồ / Nav2) nên sai số tích luỹ theo thời gian; an toàn va chạm dựa vào sân trống, kích thước hành vi nhỏ và lớp kiểm tra LiDAR theo hướng chạy (không áp dụng khi quay tại chỗ). Khi mô phỏng chạy chậm hơn thời gian thực, hành vi kéo dài hơn nhưng quãng đường và góc quay vẫn đúng vì được đo bằng odometry.